Practice Problem 4 Guide Questions

1. Why is it important to check whether the input character is a letter before performing arithmetic on it? What would happen if we skipped this step?

Because every character has an ASCII code associated with it, we need to ensure we are actually working with a letter before performing arithmetic on it so we get expected output. If we didn't check, we could potentially get numbers outside of the 1-26 range.

2. How can you use the numeric relationship between characters in C to determine a letter's position when the alphabet is read backward? Test your approach with 'a', 'd', and 'z'. What pattern do you observe?

The ASCII table is laid out in a way where both uppercase and lowercase letters are all placed next to each other, with each letter being one more than the previous one. This means that when we subtract letters, the difference is basically telling us how far away the letters are from each other in a position-sense. So, if we simply subtract using 'A' or 'Z', the difference gives us the letter's position in the alphabet as a whole (0-indexed).

'a' - 'a' = 0 (first letter of the alphabet)
'd' - 'a' = 3 (fourth letter of the alphabet)
'z' - 'a' = 25 (26th letter of the alphabet)

3. What happens to the for loop if you instead initialize i = 0 and use i < T as the condition? How would it affect the number before the : in each case? Rewrite the loop in four different ways by combining: (1) initializing i to either 0 or 1; and (2) using either < or <= as the condition.

If we used "i = 0" and "i < T", we would get the same number of iterations. We would just have to alter our print statement slightly to account for starting at 0 instead of 1.

```
for (int i = 0; i < T; i++)
for (int i = 1; i <= T; i++)
for (int i = 0; i <= T - 1; i++)
for (int i = 1; i < T + 1; i++
```

4. What happens if you enter more than one character (e.g., "AB") on a single test case line? How does scanf(" %c", &letter) handle it compared to entering one character per line? Enter this after providing the value for T.

It will take in the characters as if you entered them individually and iterate through multiple loops automatically. So in short, if AB was typed, the loop would have iterated 2 times.

5. Rewrite the program using a while loop instead of a for loop. What changes are needed? Does it affect the program's behavior? What are the pros and cons of each style?

We can accomplish the same behavior using a while loop, but we would need to create our own incrementer variable and manually increment it every time input is received. We would break out once our incrementer is greater than the number of test cases given to us by the user.

This works, but it puts the responsibility of keeping track of which loop we are on on the user, which can open up more opportunities for error. The for loop is nicer in this situation since it will keep track of which loop we are on for us.